

Vista Food App

SDLC Model Selection & Use Case Diagram

Software Engineering Project | Bangladesh University of Professionals

Group Members

• Abrar Zahin	23524202100
• Abid Rejwan	23524202148
• Abdullah Bin Towhid	23524202054
• Tabida Ahmed	23524202134

April 2026

1. SDLC Model Selection

1.1 Project Overview

Vista Food App is a web-based food ordering system developed for Bangladesh University of Professionals (BUP). It allows students and staff to browse menu items, manage a cart, and pay via a wallet-based system. Administrators can manage food items, orders, and wallet balances. The project has moderate complexity, evolving UI requirements, and a medium-sized development team of four members.

1.2 SDLC Model Comparison Matrix

Each criterion is scored 1–5 (5 = best fit). The weighted score is calculated by multiplying the criterion priority weight by the model score.

Criterion	Priority Weight	Waterfall	V-Model	Iterative	Spiral	Agile	Prototype
Requirement Clarity	4	5	4	3	2	3	2
Flexibility to Change	4	2	3	5	4	5	4
Customer Involvement	3	2	2	4	3	5	3
Risk Management	4	2	4	3	5	4	3
Team Size (Small)	3	4	3	4	2	5	3
Delivery Speed	4	2	2	4	3	5	3
Testing Rigor	3	3	5	3	4	4	3
Budget Constraint	3	4	3	3	2	5	3
Documentation	2	5	5	3	4	3	4
Scalability Support	3	3	3	4	4	4	3
TOTAL SCORE		102	110	120	109	143	101

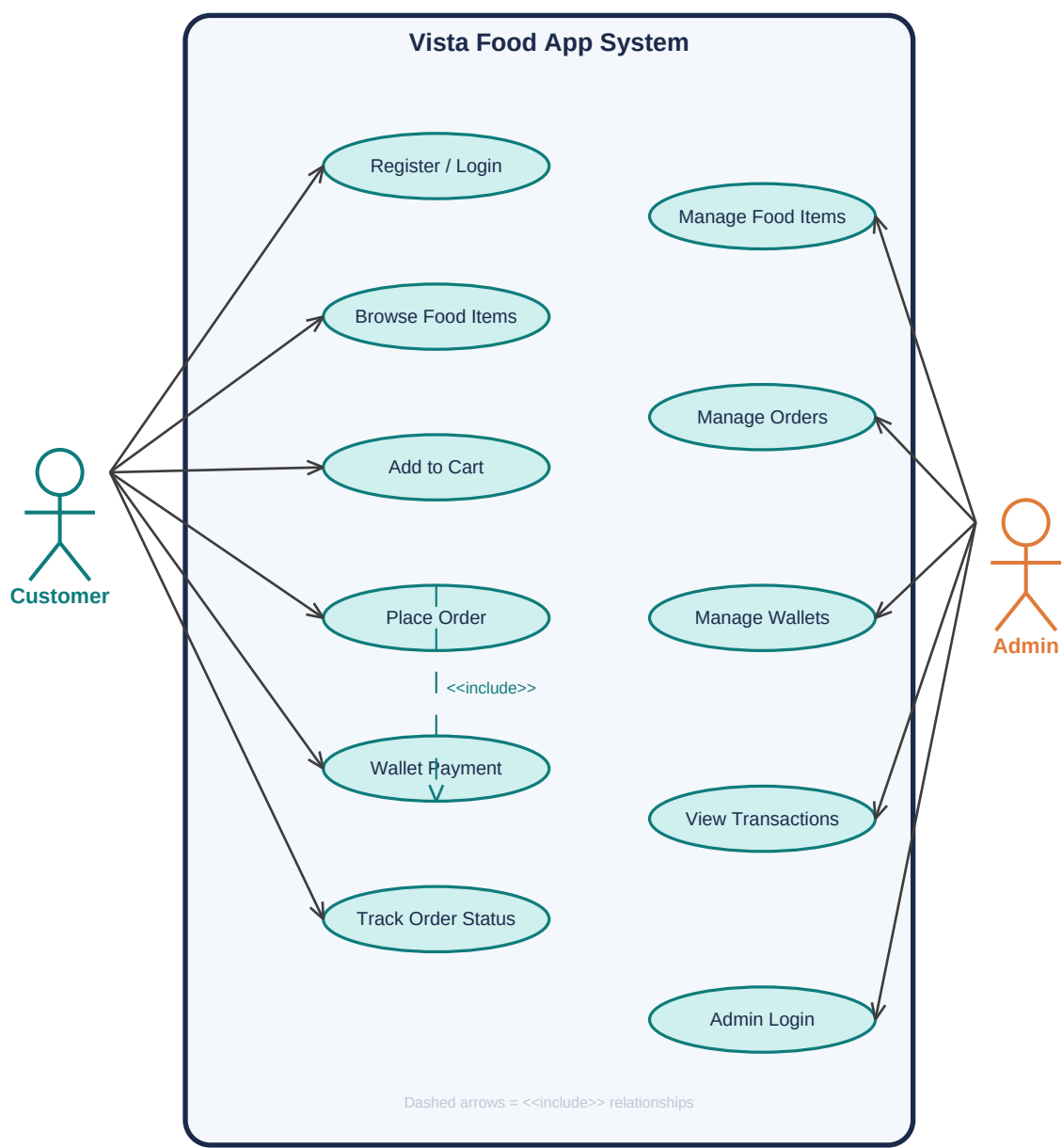
1.3 Selected SDLC Model: Agile

Waterfall 102	V-Model 110	Iterative 120	Spiral 109	Agile 143	Prototype 101
-------------------------	-----------------------	-------------------------	----------------------	---------------------	-------------------------

- **Evolving Requirements:** Vista Food App has a UI and feature set that is expected to evolve through user feedback. Agile accommodates changing requirements at every sprint.
- **Short Iterations / Fast Delivery:** Agile sprints (1–2 weeks) allow the team to deliver working increments quickly, e.g., auth module first, then cart, then payment.
- **Customer / Stakeholder Involvement:** Continuous feedback from BUP users and admins can be incorporated after every sprint review.
- **Small Team Compatibility:** Agile is highly effective for teams of 2–6 people, making it ideal for this four-member group.
- **Risk Mitigation:** Regular sprints expose integration issues early rather than at the end of the project.
- **Modern Tech Stack Alignment:** React + Node.js + MongoDB maps naturally onto iterative feature-based development cycles used in Agile.

2. Use Case Diagram

The following Use Case Diagram illustrates the interactions between the two primary actors (Customer and Admin) and the Vista Food App system. Each use case corresponds to a functional requirement defined in the SRS.



Customer	Register/Login, Browse Food Items, Add to Cart, Place Order, Wallet Payment, Track Order Status
Admin	Admin Login, Manage Food Items, Manage Orders, Manage Wallets, View Transactions

3. Use Case Descriptions

UC-01 – Register / Login	Actor: Customer
Description: User creates an account with name, email, and password, or logs in to access features.	Main Flow: 1. User submits registration/login form. 2. System validates credentials. 3. Session token issued on success.
UC-02 – Browse Food Items	Actor: Customer
Description: System displays all available food items with image, name, description, and price.	Main Flow: 1. User opens the menu page. 2. System fetches items from database. 3. Items displayed in a responsive grid.
UC-03 – Add to Cart	Actor: Customer
Description: User selects items and manages quantity in the shopping cart.	Main Flow: 1. User taps "Add to Cart" on an item. 2. Cart updated in real time. 3. Total price recalculated.
UC-04 – Place Order	Actor: Customer
Description: User submits the cart as a confirmed order if wallet balance is sufficient.	Main Flow: 1. User clicks "Place Order". 2. System checks wallet balance. 3. Order created; wallet deducted (<> Wallet Payment).
UC-05 – Wallet Payment	Actor: Customer
Description: System automatically deducts the order total from the user's wallet.	Main Flow: 1. Triggered by Place Order (<>). 2. System deducts amount from wallet. 3. Transaction recorded.
UC-06 – Track Order Status	Actor: Customer
Description: Customer can view real-time order status updates.	Main Flow: 1. User opens order history. 2. System shows current status (Pending / Preparing / Delivered). 3. Socket.IO pushes live updates.
UC-07 – Manage Food Items	Actor: Admin
Description: Admin can add, edit, or delete food items from the menu.	Main Flow: 1. Admin accesses the food management panel. 2. Fills in item details and uploads an image. 3. Item saved to database and visible to customers.
UC-08 – Manage Orders	Actor: Admin

Description: Admin reviews all orders and updates their status.	Main Flow: 1. Admin views the order dashboard. 2. Selects an order and changes status. 3. Customer notified via real-time update.
UC-09 – Manage Wallets	Actor: Admin
Description: Admin can add or deduct wallet balance for any user.	Main Flow: 1. Admin searches a user account. 2. Enters credit or debit amount. 3. Wallet balance updated and transaction logged.
UC-10 – View Transactions	Actor: Admin
Description: Admin reviews full transaction history across all users.	Main Flow: 1. Admin opens transaction logs. 2. Filters by date, user, or amount. 3. Exports report if needed.
UC-11 – Admin Login	Actor: Admin
Description: Admin authenticates through a secure, separate login portal.	Main Flow: 1. Admin enters credentials. 2. System verifies admin role. 3. Access granted to admin dashboard.